SCARA · Documento maestro v2

Versión final · 01/11/2026. Reemplaza como referencia de trabajo al documento maestro v1 (`00-SCARA-Master.pdf`, 02/05/2026), que se conserva sin cambios. La primera versión, del 30/10/2026, llevó a clase las dos propuestas de la sección 1. Esta versión describe el sistema implementado en la ronda de teach & play (issues #5 a #18), con la evidencia de la campaña de validación (sección 10) y las 76 decisiones del registro, D0–D75, cada una con su porqué (sección 11).

Cómo leer el estado de cada parte:

Marca	Significado
Decidido	El grupo lo decidió; está en el registro de decisiones y, si es difícil de revertir, en un ADR.
Propuesta al profesor	La implementamos igual y se revisa en clase. Si el profesor la cambia, se cambia la configuración, no la arquitectura.
Evidencia	Ensayo en Webots registrado, con su versión y su tolerancia. Prueba el diseño simulado, no un robot construido.
Pendiente	Lo que no se pudo verificar: la placa real y la compilación en macOS.
Parámetro provisional	Valor con fuente o hipótesis explícita, todavía no confirmado. No es una medición.

Contenido

1. Propuestas para revisar en clase (D9, D10/D11)
2. Cambios respecto de la v1 y fuentes
3. Mecánica vigente
4. Transmisión y cinemática híbrida
5. Electrónica
6. Protocolo `scara/1`
7. Homing y seguridad
8. Teach & play
9. Qué se simula y qué no
10. Validación
11. Registro de decisiones

1. Propuestas para revisar en clase

Las dos propuestas ya se están implementando: el profesor puede aceptarlas, corregirlas o pedir otra cosa. Cada una tiene una línea para anotar la respuesta.

P1 · Microstepping 1/8: 1600 pulsos por vuelta (D9)

Qué dice el maestro v1. Los cuatro TB6600 configurados a 400 pulsos por vuelta (medio paso), p. 14.

Qué proponemos. Configurar los TB6600 a 1/8 de paso, 1600 pulsos por vuelta de motor, con sus llaves DIP. El valor es un único parámetro del firmware (`firmware/config.h`, D44), que comparten el Arduino y la planta simulada: cambiarlo no toca la URC ni el protocolo, que habla en unidades articulares (ADR 0003).

Por qué. Con 400 pulsos, un pulso de M1 mueve J1 0,225°. Con el brazo estirado (500 mm del eje de J1 al punto de herramienta) eso es casi 2 mm en el extremo, y un tramo LIN se vería escalonado. Con 1/8 baja a medio milímetro.

Resolución por pulso, con motores de 200 pasos por vuelta (1,8°), reducción 4:1 en J1–J3 y tornillo ACME de 8 mm por vuelta en J4:

Configuración	Pulsos por vuelta de motor	J1–J3 por pulso	Extremo, brazo estirado (500 mm)	Extremo, solo J2 (250 mm)	J4 por pulso
1/2 (maestro v1)	400	0,225°	1,96 mm	0,98 mm	0,020 mm
1/8 (propuesta)	**1600**	**0,05625°**	**0,49 mm**	**0,25 mm**	**0,005 mm**
1/16	3200	0,028°	0,25 mm	0,12 mm	0,0025 mm

Costo: más pulsos por segundo. El UNO genera los pulsos de los cuatro motores. Frecuencias con las velocidades máximas de los motores (D76): J1 y el antebrazo (J1 + J2) a 90 °/s, J3 a 180 °/s, J4 a 40 mm/s. M2 sigue al antebrazo por el acople (sección 4), así que su límite es el de J1 + J2.

Configuración	M1	M2 (J1 + J2)	M3	M4	Total
1/2	400 Hz	400 Hz	800 Hz	2000 Hz	≈ 3,6 kHz
1/8	**1600 Hz**	**1600 Hz**	**3200 Hz**	**8000 Hz**	**≈ 14,4 kHz**
1/16	3200 Hz	3200 Hz	6400 Hz	16000 Hz	≈ 29 kHz

Como referencia, GRBL genera hasta unos 30 kHz de pulsos en un UNO con interrupción de timer. Con 1/8 queda margen. Con 1/16 se llega a ese techo sin una mejora visible: el extremo ya se mueve medio milímetro por pulso. La velocidad «1000 rpm» del maestro no es alcanzable con ninguna de las tres: a 1/8 serían 26,7 kHz por motor. Por eso las velocidades se acotan en el firmware.

Cómo quedó (D44, D45, D48, D76). `SCARA_MICROSTEPS` vale 8 en `firmware/config.h`, y las pruebas compilan la conversión con los seis valores del TB6600. Los límites se fijaron por motor, con el doble del par que piden (sección 9). Con 1/8, el que más pide es M4, 8000 pulsos/s, y el Timer1 del UNO da hasta 10

000 por motor (sección 5); con 1/16, M4 pediría 16 000 y el sketch no compila hasta subir el tick. La campaña de validación corrió con 1/8.

Lo que no prometemos. El microstepping mejora la suavidad y la resolución ordenada. La exactitud real depende del par por micropaso, de la carga y de las correas. No se presenta 0,5 mm como precisión del robot.

Descartado. 400 pulsos (resolución gruesa para rectas) y 1/16 o más (exige más pulsos de los que conviene pedirle al UNO, sin beneficio visible).

Respuesta del profesor: _____

P2 · Sensores Hall: ubicación y signos de las referencias (D10, D11)

Qué dice el maestro v1. «Sensores de inicio y final de carrera en los cuatro ejes», con imanes de neodimio y sensores de efecto Hall, sin encoders (p. 14). No dice cuántos sensores hay, dónde van ni con qué polaridad; la sección «Sensor Hall» está vacía. El CAD trae un `KY-003` que no está colocado en ningún ensamble y no hay pieza de imán.

Qué proponemos. Un sensor KY-003 por articulación, en la pieza fija de esa articulación, y dos imanes en la pieza que se mueve: uno en el inicio de carrera (referencia de homing) y otro en el fin (protección por sobrerrecorrido). El firmware distingue los dos imanes por el sentido de giro.

Sensor	Fijo en	Imanes en	Mide	Extremo de referencia del homing
H1	Torre	Brazo, cerca de J1 = −135° y +135°	J1	**−135°**
H2	Brazo, en el codo	Antebrazo, cerca de J2 = −135° y +135°	J2 **relativo**	**−135°**, mismo signo que J1
H3	Antebrazo	Polea de muñeca, cerca de J3 = −150° y +150°	J3	Cualquiera de los dos; la implementación eligió **−150°** (D56)
H4	Cuerpo Z	Pistón, arriba y abajo	J4	**Arriba** (J4 = 0 mm)

Convención de signos: la del modelo actual (`scara/kinematics.py`). J1–J3 son positivos en sentido antihorario visto desde arriba; J4 es positivo hacia abajo. Los límites articulares son parámetros provisionales: ±135° en J1 y J2, ±150° en J3 y 0–120 mm en J4.

Por qué.

- **Un sensor por eje** entra en el Arduino UNO: 15 de 20 pines por USB (sección 5). Dos sensores por eje con entradas propias no entran.
- **H2 en el codo, midiendo J2 relativo.** Los topes que importan son los relativos: cables en los carreteles y choque entre brazo y antebrazo. Medir desde la base (sobre la polea de M2) daría el ángulo absoluto del antebrazo y no protegería esos topes. Los cables de M3 y M4 ya pasan por el brazo, así que llevar uno más al codo no agrega dificultad.
- **J1 y J2 se referencian en el extremo negativo, los dos con el mismo signo.** El homing busca J1 primero y queda en −135°. Si después J2 buscara en +135°, el codo plegado hacia la torre chocaría: con el chequeo de colisiones del repositorio, el soporte Z toca la torre. Con los dos en −135°, el recorrido completo desde la pose de reposo queda libre. Esta conclusión vale con los límites provisionales y las envolventes de colisión simplificadas.
- **J4 arriba** aleja la herramienta de la mesa antes de mover los demás ejes.

- **En Webots**, el imán es un `Emitter` de radio de alcance corto (unos 5 mm) y el KY-003 un `Receiver`. La activación sale de la geometría del modelo, no de leer la posición perfecta del simulador, que es lo que hacía el banco provisional `scara-dev/1`. Al implementarlo, Webots resultó no entregar paquetes dentro de un mismo robot: la planta mide la distancia entre los dos nodos, que es la misma regla (D57, sección 3).

Secuencia de homing propuesta (D11). Arranca desde la pose de reposo y termina volviendo a ella.

1. J4 sube hasta su Hall.
2. J1, moviendo M1 y M2 la misma cantidad de pasos: así J2 no cambia mientras se busca J1.
3. J2, con M2 solo: con M1 quieto, M2 cambia únicamente J2.
4. J3, con M3.

En cada eje: avance rápido hasta el Hall, retroceso hasta que se apague y aproximación lenta, siempre desde el mismo lado. La búsqueda está acotada en pasos y en tiempo; si el Hall no aparece o está trabado, el homing aborta sin declarar referencia.

Descartado. Medir J2 desde la base; `TouchSensor` en Webots (genera choques físicos); calcular el Hall con la posición del simulador; buscar J2 en el extremo positivo.

Cómo quedó (D56, D72). Los imanes están centrados en los extremos, con una zona activa de ±5° (±3 mm en J4). La referencia es el flanco del lado alcanzable, 5° antes del tope: J1 y J2 en −130°, J3 en −145° y J4 a 3 mm de arriba. El homing está en la sección 7 y su evidencia, en la sección 10.

Respuesta del profesor: _____

2. Cambios respecto de la v1 y fuentes

Regla de fuentes. En la clase del 27/10/2026 el profesor indicó que los SolidWorks son la fuente de verdad y que el maestro v1 está desactualizado. Cuando dos fuentes difieren, gana el CAD; cuando el CAD no alcanza, se usa el maestro v1, y la diferencia queda documentada. Ningún dato se presenta como medido: no hay robot físico (ADR 0001).

Tema	Maestro v1 (02/05/2026)	Maestro v2	Origen
Motores	2 NEMA 23 + 2 NEMA 17	Igual. El ensamble guarda un `NEMA23-2` sin ubicación: es una instancia suprimida o residual	D20
Microstepping	400 pulsos/vuelta	1600 pulsos/vuelta, configurable	D9 · propuesta
Sensores Hall	Inicio y fin en los cuatro ejes, sin ubicación	Uno por eje con dos imanes, ubicación y signos definidos	D10, D11 · propuesta
Recorrido de M2	Se trata J2 como M2/4 (p. 8)	M2 fija el ángulo absoluto del antebrazo: J2 = (θM2 − θM1)/4. M2 recorre hasta 540° de salida	Sección 4
Velocidad rotacional	Misma celda con 4,44°/s y 266 rpm (p. 8)	No son equivalentes: 4,44 son pulsos por grado. No se usa esa celda	Transmisión
Protocolo	Capítulo sin contenido	`scara/1`, texto por línea, TCP	D4–D6, D24, D27, D35, D40–D43, D50
Software	Capítulo sin contenido	URC en la PC, firmware C++ único en la UCI	D3, D6
Homing y pose de reposo	No definidos	Secuencia J4, J1, J2, J3; reposo con el brazo estirado	D11, D30, D56, D72
Teach & play	No existía	Jog desde la URC, gestor de trayectorias, tramos LIN y PTP, reproducción	D12–D14, D28–D34, D62–D63, D68–D69, D75
Modelo de motores	No existía	Pasos, transmisión y acople (N1); par limitado y pérdida de pasos (N2)	D8, D44, D64
Validación	No existía	Campaña en Webots de los escenarios de homing, teach & play y fallas F1–F5	D25, D73

Discrepancias mecánicas todavía abiertas. Las diferencias entre el maestro, los planos sueltos y el CAD (cuerpo Z, pistón, varilla ACME, buje exterior, carreteles A, polea de muñeca, transmisión de M3, ejes de la torre) están en la revisión para el profesor del 27/10/2026 y en Datos para el profesor. Ninguna cambia las propuestas de este documento.

Fuentes. Consigna del TP final (rev. 13/10/2026); maestro v1; planos y ensambles de `Modelos3D/`; clase del 27/10/2026; investigación del 29/10/2026; registro de decisiones D0–D75; ADR 0001–0007; glosario.

3. Mecánica vigente

SCARA de cuatro articulaciones: J1 (hombro), J2 (codo) y J3 (muñeca) rotacionales; J4 prismática vertical.

Dato	Valor	Estado
Distancia J1–J2	250 mm entre ejes	Del CAD
Distancia J2–J3	250 mm entre ejes	Del CAD
Recorrido J1, J2	±135° (270°)	Parámetro provisional, coincide con el maestro
Recorrido J3	±150° (300°)	Parámetro provisional, coincide con el maestro
Recorrido J4	0–120 mm, positivo hacia abajo	Parámetro provisional, coincide con el maestro
Punto de herramienta	Centro de la cara exterior de `Z-SoporteEfector`, sin pinza	Provisional hasta que se defina la herramienta
Envolventes de colisión	Cajas simplificadas por cuerpo rígido	Provisional; cubren ciertos pares, no todos los contactos internos
Masas e inercias	Calculadas del CAD con densidades nominales	Provisional (propiedades)
Sensores Hall	Un KY-003 por articulación, en la pieza fija, y dos imanes en la que se mueve, centrados en los extremos del recorrido, con una zona activa de ±5° (±3 mm en J4). Radios desde el eje: 60, 40 y 30 mm en J1–J3	Propuesta al profesor (D10, D56); en el modelo de Webots desde el issue #8

El montaje del modelo y sus elecciones están en Montaje y autocolisiones y Ensamble CAD.

Hall en Webots (D57). El KY-003 es un `Receiver` y el imán un `Emitter` con su alcance. Webots no entrega paquetes dentro de un mismo robot, así que la planta mide la distancia entre los dos nodos y activa el Hall cuando el imán está más cerca que su alcance. La activación sale de la geometría del modelo: si se corre un imán, se corre el flanco.

El chequeo de colisiones deja libre el camino físico completo del homing y de la vuelta al reposo, medido en Webots (validación del homing). Vale con las envolventes provisionales.

4. Transmisión y cinemática híbrida

Reconstruida de las listas de componentes de los ensambles y de los planos. Certeza alta en la topología y las relaciones; los signos y cruces de correa los fijamos como convención.

- **M1** (base) → polea 20T → correa → 80T sobre el tubo exterior `A-Eje`, abulonado al brazo. Mueve J1 con reducción 4:1.
- **M2** (base) → 20T → 80T sobre el eje interior `A-EjeInterior`, que pasa por dentro del tubo. Arriba lleva una polea 60T; una correa a lo largo del brazo la une con otra 60T sobre `B-Eje`, fijo al antebrazo.
- **M3** (antebrazo) → 20T → 80T en la muñeca. Sin acople.
- **M4** (columna Z) → acople flexible → varilla ACME de 8 mm por vuelta. Sin acople con J3.

Con θ el ángulo del eje de cada motor y los ceros alineados:

```
J1 = θM1 / 4
J2 = (θM2 − θM1) / 4          (porque J1 + J2 = θM2 / 4)
J3 = θM3 / 4
J4 = 8 mm · vueltas de M4
```

Consecuencias que se ven en la simulación:

1. **M2 fija el ángulo absoluto del antebrazo, no J2.** Si gira M1 y M2 queda quieto, el brazo gira, el antebrazo conserva su orientación en el mundo y J2 cambia en sentido opuesto. Es lo que señaló el profesor: mover el motor del hombro mueve hombro y codo.
2. **Para mover solo J1, M1 y M2 dan la misma cantidad de pasos.** El firmware resuelve ese acople; la URC habla en articulaciones (D5).
3. **El rumbo del extremo depende solo de M2 y M3:** ψ = J1 + J2 + J3 = (θM2 + θM3)/4.
4. **M2 recorre más de lo que dice el maestro.** Con J1 y J2 en ±135°, el antebrazo cubre 540° absolutos: 9600 pulsos a 1/8 (2400 a 400 pulsos/vuelta), no 1200.

Conversión a pulsos con 1/8 (6400 pulsos por vuelta de articulación): 17,78 pulsos por grado en J1–J3 y 200 pulsos por mm en J4. Los pulsos de M2 salen de J1 + J2. El firmware lo hace con enteros: 8 pasos cada 45 centésimas de grado y 2 pasos por centésima de mm.

Límites de los motores (D45, D76). La limitación física está en los motores, no en las articulaciones. Se fijan por motor y se derivan con el acople: M1 limita a J1, y M2 limita a **J1 + J2**, el ángulo absoluto del antebrazo. Con M1 y M2 a 360 °/s, J1 llega a 90 °/s; si J1 gira a 90 °/s, J2 puede ir de 0 a −180 °/s. M3 deja a J3 en 180 °/s y M4 a J4 en 40 mm/s. Se eligieron con las curvas de par de N2 (sección 9): con la corriente nominal y 5 kg en la herramienta, el par que piden queda al menos 2 veces por debajo del disponible en M1–M3, y 1,4 en M4, que lo pone el peso.

Par de los motores (N2, D64). Cada motor tiene el par que le da su curva par–velocidad, reflejado por la transmisión:

Motor	Par de retención	A baja velocidad	Velocidad de esquina	Fuente
M1, M2	0,80 N·m	0,56 N·m	1308 °/s	Familia Minebea 23KM-K3 (23KM-K349B)
M3	0,31 N·m	0,22 N·m	3375 °/s	STP-42D201-37, de la misma serie
M4	0,20 N·m	0,14 N·m	7714 °/s	Supuesto: NEMA 17 corto a 0,7 A

Las hojas de datos exactas no aparecieron; las fuentes y los supuestos de cada parámetro (inercia, inductancia, eficiencias del ACME y la correa) están en Transmisión.

Jog por motor (D46). Un modo de prueba de la URC mueve un solo motor. Es la forma de mostrar el punto 1 en la defensa.

Evidencia (N1 en Webots, 30/10/2026). Un supervisor aparte midió la posición física simulada contra la estimada por pasos (informe):

Orden	Cambio físico simulado
Jog articular de J1	J1 +6,24°; J2, J3 y J4 sin cambio
Jog por motor de M1	J1 +6,24°, J2 −6,24°; rumbo de la herramienta sin cambio

Orden	Cambio físico simulado
Jog lento de J3	Escalones de 0,05625°: un paso de 1/8

Detalle, tabla por microstepping y contraste con los planos: Transmisión.

5. Electrónica

Componente	Dato	Estado
UCI	Arduino UNO: 2 KB de RAM, 32 KB de flash, 20 pines de E/S, gabinete separado del brazo	Del maestro (D18)
Enlace	USB-serie a 115200 baudios, puenteado a TCP con socat. Alternativa: shield Ethernet	D1, D18
Drivers	Un TB6600 por motor	Del maestro
Microstepping	1/8, 1600 pulsos por vuelta	Propuesta (P1, D9)
M1, M2	Minebea 23KM-K381-P1, NEMA 23, 1,8°	Del maestro; corriente y par de la familia 23KM-K3 (D64)
M3	Shinano Kenshi STP-42D221-03, NEMA 17	Del maestro; par del STP-42D201-37, corriente supuesta (D64)
M4	42D265, NEMA 17, 1,9 V, 0,7 A	Del maestro
Sensores	4 × KY-003 (Hall digital) y 8 imanes	Propuesta (P2, D10)
Alimentación	36 V 10 A para drivers; 12 V 2 A para Arduino y ventiladores	Del maestro. N2 calcula el par con 24 V en los TB6600, que es más conservador (D64)

Presupuesto de pines del UNO (D18).

Uso	Pines
STEP + DIR × 4 motores	8
ENABLE común a los cuatro drivers	1
Hall: una entrada por eje	4
Subtotal	**13**
Serie por USB (D0, D1)	2
Total por USB + socat	**15 de 20**
Con shield Ethernet (SPI D10–D13 + D4)	20 de 20, sin margen

Asignación de pines (D47). Deja libres D4 y D10–D13, los del shield Ethernet: pasar a Ethernet no obliga a recablear.

Pines	Uso
D2, D3, D5, D6	STEP de M1–M4 (PORTD: los pulsos de M1 y M2 salen juntos)
A0–A3	DIR de M1–M4
D7	ENABLE común (ENA+ de los TB6600, cátodo común; en alto, sin corriente)

Pines	Uso
D8, D9, A4, A5	Hall de J1–J4 (KY-003 con pull-up interno; activo en bajo)

Pasos por timer (D48). El núcleo cuenta los pasos; la interrupción del Timer1, a 20 kHz, los convierte en pulsos con los tiempos del TB6600 (DIR estable antes del flanco, 50 µs en alto): hasta 10 000 pasos/s por motor, contra los 8000 que pide J4 a 40 mm/s. Si se juntan pasos pendientes, el firmware espera en lugar de descartarlos.

Compila sin placa. Con `arduino-cli` y el mismo núcleo que corre en Webots, con el protocolo completo, el homing, la cola de segmentos y `EV LOST`: 16 148 bytes de flash (50 % de 32 256) y 1297 bytes de RAM global (63 % de 2048), sin avisos en el código del proyecto. La prueba automática exige que la RAM global no pase del 75 %, para dejar lugar a la pila.

socat probado con un puerto serie virtual (D49). La URC opera el firmware, compilado para la PC, por un puerto serie virtual y socat, con la misma receta que se usará con el USB del UNO y sin cambios en la URC: arranque con `BOOT`, habilitar, referencia manual, jog de J1 con J2 quieto, deshabilitar, desconexión y nuevo `BOOT` al reabrir el puerto. Receta, cableado y resultados: UCI en el Arduino UNO. Con la placa real no se probó: no hay hardware.

Sin corriente, J4 cae. La varilla ACME de 4 hilos y paso largo probablemente no es autobloqueante. Por eso la pose de reposo tiene J4 abajo, y la falla F2 (driver sin corriente) se demuestra en J4.

6. Protocolo `scara/1`

La URC y la UCI se comunican con un protocolo de texto propio, al nivel de un Arduino. El día del robot real, Webots se reemplaza por el Arduino sin cambiar la URC.

Roles (D1). La URC escucha y la UCI se conecta: el controlador de Webots, socat con el Arduino por USB, o un Arduino con Ethernet. La URC queda siempre esperando; el robot puede reiniciarse o reconectarse sin tocarla. Puerto 9000 para `scara/1` y 9001 para el canal de instrumentación, los dos configurables (D24).

Formato (D4). ASCII, una línea por mensaje terminada en LF (se ignora CR), de hasta 64 bytes, campos separados por espacios y solo enteros. Se puede hablar a mano con `nc` o el monitor serie.

Unidades (D5, ADR 0003). Centésimas de grado en J1–J3 (`9000` = 90,00°) y centésimas de mm en J4. El firmware convierte a pasos y resuelve el acople M1/M2.

Reparto (D6, ADR 0005). La URC resuelve la cinemática inversa, las rectas y el perfil de velocidad, y manda segmentos cortos con duración. El firmware los encola, los ejecuta con su reloj y responde `OK` cuando hay lugar en la cola. Es la división de GRBL y Klipper.

Sentido	Mensaje	Qué hace
URC → UCI	`HELLO`	Pide identificación
URC → UCI	`PING`	Latido para el watchdog
URC → UCI	`ENABLE` / `DISABLE`	Habilita o corta los drivers; la UCI arranca sin corriente y `DISABLE` pierde la referencia (D40)
URC → UCI	`HOME [j]`	Homing de todos los ejes o de uno
URC → UCI	`ZERO`	Referencia manual desde la pose de reposo
URC → UCI	`JOG j ± vel` / `JOG STOP`	Jog articular local: el de sin referencia (D58)
URC → UCI	`JOG Mm ± vel`	Jog por motor: mueve solo el motor `m`, sin compensar el acople (D46)
URC → UCI	`SEG n ms q1 q2 q3 q4`	Encola el segmento n: llegar a esa configuración articular en ms milisegundos
URC → UCI	`STOP` / `HALT`	Frena y conserva la referencia / parada inmediata, que la pierde si había movimiento (D41)
URC → UCI	`STATUS`	Pide un estado inmediato
UCI → URC	`BOOT`	Arranque o reinicio: «no estoy referenciado»
UCI → URC	`HI`	Respuesta a `HELLO`, con versión de firmware y de protocolo

Sentido	Mensaje	Qué hace
UCI → URC	`S`	Estado, 20 por segundo: modo, ejes referenciados, posición estimada por pasos, bits de Hall, lugares libres en la cola, último segmento terminado
UCI → URC	`OK n` / `ERR código palabra`	`OK n`: el segmento n entró en la cola (control de flujo). `ERR`: orden rechazada o evento que frena, con códigos estables. Las demás órdenes no responden si salen bien: el efecto se ve en `S` (D35)
UCI → URC	`EV HOMED j` / `EV LOST j pasos`	Eje referenciado / diferencia de pasos detectada al cruzar un Hall

Códigos de `ERR` (D35, D42). Estables: se agregan al final y no se renumeran. 1 `comando`, 2 `argumentos`, 3 `largo`, 4 `watchdog`, 5 `ocupado`, 6 `deshabilitado`, 7 `referencia`, 8 `velocidad`, 9 `cola`, 10 `fin`, 11 `ausente`, 12 `trabado`.

Arranque y reconexión (D36, D40, D43). La UCI arranca sin corriente (modo `OFF`) y sin referencia, y anuncia `BOOT`. Todo `BOOT` deja al robot sin referencia en la URC; uno a mitad de una conexión es la falla F5. En Webots, el firmware arranca con la primera conexión, como el UNO que se reinicia cuando socat abre el puerto, y no se reinicia si después se corta la red.

Valores iniciales (D27). PING cada 250 ms; watchdog de 1 s; segmentos de 20 ms; cola de 16 segmentos (tres durante el jog verificado, D75); 115200 baudios. Caben con holgura en un UNO: a 50 segmentos por segundo son unos 2 KB/s, contra unos 11 KB/s del enlace serie.

Canal de instrumentación (D2, ADR 0004). Lo que solo existe en la simulación va por otro puerto: posición física simulada, inyección de fallas, carga y recarga del mundo. Así la URC no puede depender sin querer de información que el robot real no da.

Especificación y conformidad (D42). La especificación completa, con códigos de error, campos de `S` y conversaciones de ejemplo, está en `PROTOCOLO_SCARA_1.md`. Las mismas conversaciones de conformidad se ejecutan como pruebas contra el firmware (compilado en la PC) y contra la URC (con una UCI falsa por TCP), así los dos lados no pueden entender el protocolo de maneras distintas. El ensayo en Webots está en UCI en Webots.

7. Homing y seguridad

Pose de reposo (D30). J1 = J2 = J3 = 0° (brazo estirado) y J4 abajo. Se deja ahí al apagar, con el botón «Estacionar», y desde ahí arranca el homing. Contraída (J2 = ±135°) toca la torre incluso quieta; estirada queda libre y J1 puede barrer hasta los dos extremos.

Homing (D11, D56, D72). Implementado en el issue #8 y ajustado tras la QA de la ronda 3 (PR #33). Antes de empezar, la URC muestra un dibujo de la pose de reposo y pide confirmarla. `HOME` busca J4, J1 (M1 y M2 juntos), J2 (M2 solo) y J3. En cada eje:

1. Si el Hall ya está activo, sale del imán a 1 °/s (desde el reposo, J4 descansa sobre el de abajo).
2. Busca a 15 °/s en J1–J3 y a 10 mm/s en J4 hasta que el Hall se activa.
3. Retrocede a 1 °/s hasta que se apaga, y 3° (3 mm) más.
4. Vuelve a 1 °/s al mismo flanco, que fija la referencia: J1 y J2 en −130°, J3 en −145° y J4 a 3 mm de arriba. El firmware envía `EV HOMED j`.

La búsqueda está acotada en recorrido y en 60 s por eje. Un Hall que no aparece (`ERR 11 ausente`) o que no se apaga (`ERR 12 trabado`) aborta el homing sin declarar referencia. Con los cuatro ejes encontrados, la URC vuelve sola a la pose de reposo con un tramo PTP al 100 %, verificado contra choques; «Estacionar» es el mismo tramo. El homing tarda unos 55 s y la vuelta, unos 14 s.

Por qué esas velocidades (D72). N2 verifica que la búsqueda arranca y frena sin rampa sin perder pasos, aun con 5 kg; J4 a 15 mm/s sí pierde pasos subiendo 5 kg, y por eso va a 10 mm/s. La aproximación lenta, que es la que fija la referencia, no cambió. El homing se puede repetir después de frenarlo: el firmware recuerda qué ejes tienen una cuenta de pasos que sale de una referencia, porque con `STOP` no se pierden pasos.

Paso	J4	J1	J2	J3
Motores	M4	M1 y M2 juntos	M2	M3
Sentido de búsqueda	Arriba	Negativo	Negativo	Negativo
Flanco de referencia	3 mm	−130°	−130°	−145°

Referencia manual. Botón con confirmación que declara referenciada la pose actual, después de llevar el robot con jog hasta las marcas de la pose de reposo. Es menos precisa que el homing.

Sin referencia (D21, D58). El firmware baja todo jog a 5 °/s (5 mm/s). El Hall hace de fin de carrera: un jog que lo activa frena solo (`ERR 10 fin`), y uno que arranca sobre un imán puede salir de él, pero no seguir hacia el tope. Capturar y reproducir quedan bloqueados: `SEG` exige referencia.

Con referencia (D28, D75). La URC planifica también el jog, en segmentos de 20 ms verificados contra límites, alcance, configuración del codo y colisiones: el jog frena antes de un límite o de un choque con la torre (§8).

Paradas (D40, D41, D52). `Esc` y «Stop» envían `STOP`: frena y conserva la referencia; los segmentos (un tramo o el jog verificado) frenan con rampa y el jog del firmware, que va lento, en seco. «Parada inmediata» envía `HALT`: para en seco y, si había movimiento, pierde la referencia, porque parar en seco puede perder pasos. Los dos dejan los drivers con corriente: J4 no cae. «Deshabilitar motores…» corta la corriente y pierde la referencia.

Tres frenos para el jog y watchdog (D37). La página renueva el pedido de jog cada 150 ms y frena al soltar la tecla, al perder el foco o al ocultarse. Si la página deja de renovarlo, la URC frena el jog a los 500 ms. Si la UCI pasa 1 s sin mensajes de la URC, frena sola.

Reinicio de la UCI. El firmware anuncia `BOOT` al arrancar y la URC marca la referencia como perdida. Con USB, abrir el puerto reinicia la mayoría de los Arduino: la URC nunca supone que el robot sigue referenciado tras una reconexión.

Fallas ensayadas (D23).

Falla	Qué es	Cómo se detecta sin encoders
F1	Pérdida de pasos por sobrecarga o aceleración	`EV LOST` al cruzar un Hall
F2	Driver sin corriente	J4 cae; la posición estimada por pasos ya no vale
F3	Hall trabado	El homing aborta o da un cero falso
F4	Desconexión o URC que no responde	Watchdog

Falla	Qué es	Cómo se detecta sin encoders
F5	Reinicio inesperado de la UCI	`BOOT`: referencia perdida
F6	Correa que patina o se corta	Opcional, si sobra tiempo

> **Evidencia:** el homing desde el reposo y desde un arranque con desvío, también repetido después de uno abortado, y las fallas F1–F5 están ensayados en Webots en la campaña de validación (escenarios H1, H2 y F1–F5, sección 10). El homing abortado por un Hall trabado, activo o inactivo, está además en las conversaciones de conformidad del firmware (validación del homing). F6 no se implementó.

8. Teach & play

El profesor pidió mover el robot a mano, guardar puntos, armar trayectorias con nombre y reproducirlas siguiendo rectas o el menor recorrido. AGSABOT es la referencia de la idea, no de la implementación (D0).

Jog desde la URC (D12, ADR 0007). Con el teclado de la página de la URC, estilo videojuego. Funciona igual con el robot real, que no tiene botonera.

Tecla	Jog cartesiano	Jog articular
W / S	X + / X −	J2 + / J2 −
D / A	Y + / Y −	J1 + / J1 −
R / F	Subir / bajar (Z)	J4 sube / baja
E / Q	Rumbo + / rumbo −	J3 + / J3 −
Tab	Cambiar de modo	
Mayús	Jog lento	
Espacio / Retroceso	Capturar el punto / borrar el último	
Enter / Esc	Terminar la trayectoria y pedir nombre / frenar	

En la pose de reposo, cada tecla mueve hacia el mismo lado en los dos modos. La página muestra siempre el modo, el punto de herramienta y la configuración del codo.

Jog verificado (D28, D75). Con referencia, la URC planifica el jog en segmentos de 20 ms (`urc/jog.py`) y mantiene tres en la cola del firmware. Cada uno sale de donde termina el anterior y se verifica antes de enviarlo:

- **Recorrido:** frena antes del límite de cada articulación.
- **Configuración del codo:** el jog cartesiano nunca la cambia. Desde el brazo estirado, que es una singularidad, el codo dobla hacia el lado preferido: J2 negativo por defecto (D31). Frena antes de estirar el brazo, con |J2| de 2°: el borde del alcance.
- **Choques:** frena antes de la zona de margen de la torre y del pedestal; desde adentro de esa zona, solo deja alejarse.

Al soltar la tecla, la URC deja de enviar y el robot frena al terminar lo encolado: unos 60 ms. Como el último segmento ya está verificado, no hace falta mirar adelante. La cola se cuenta en el reloj de la UCI, que la URC mide por el período del estado `s`: Webots puede correr más lento que la PC.

Jog	Normal	Lento (Mayús)
Articular con referencia	10 °/s (10 mm/s en J4)	2 °/s
Cartesiano, en el plano	20 mm/s	4 mm/s
Rumbo y altura	Como J3 y J4	Como J3 y J4
Sin referencia, solo articular (firmware, D58)	5 °/s	2 °/s

Ningún motor pasa de 10 °/s, la velocidad a la que el firmware arranca y frena sin rampa. Cerca del brazo estirado o de la torre, el cartesiano acorta el paso para respetarla. Sin referencia, el jog cartesiano está deshabilitado.

Ensayo en Webots (informe). Con la URC real contra el controlador compilado, cada jog sostenido como lo hace la página:

Caso	Resultado
Frenado al soltar la tecla	80 ms simulados de mediana, 90 como mucho, con Webots a 0,82×
Jog cartesiano en Y	Estimada a 0,06 mm de la recta, simulada a 0,17 mm; rumbo fijo
Hacia afuera desde el codo negativo	Frena con J2 = −2,03°: el codo no cambia
J4 hacia abajo	Frena en 120,00 mm, el límite del recorrido
J2 hacia el pedestal con J4 abajo	Frena a 1,14 mm, fuera de la zona de margen; después sale
Sin referencia	Cartesiano rechazado; articular a 5 °/s

Captura y puntos escritos (D13, D21, D34). Un punto de trayectoria guarda la pose (configuración articular, que incluye la configuración del codo), no el camino. Capturar exige robot referenciado y usa la posición estimada por pasos. También se puede escribir un punto en XYZ y rumbo, o en J1–J4; la URC verifica alcance, límites y choque, y dice qué falla.

Gestor de trayectorias (D29). Lista con XYZ, rumbo, tipo, velocidad y espera; reordenar, borrar, sobrescribir con la pose actual; «Ir al punto» con confirmación; guardar, guardar como, cargar y nueva, con aviso de cambios sin guardar. Las trayectorias se guardan como JSON con nombre, versión de formato, fecha, unidades y puntos.

Tramos (D14, D22). Cada tramo es LIN (el punto de herramienta recorre una recta) o PTP (todas las articulaciones se interpolan y llegan a la vez). Por defecto: LIN, 50 % de la velocidad máxima y espera de 0 s. Velocidad global de 10 % a 100 %, que no se guarda. Antes de cada play se valida la trayectoria completa: alcance, límites, velocidades, colisiones y configuración del codo. Un tramo LIN imposible se rechaza y se ofrece PTP: el robot nunca arranca un tramo que no puede terminar.

Tramo de aproximación y loop (D32, D33). Al apretar play, la URC calcula un tramo PTP desde la pose actual hasta el punto 1, al 25 % × la velocidad global, y muestra su duración. Se omite si el robot ya está ahí. Los parámetros del punto 1 rigen la vuelta del loop, del último punto al primero.

Reproducción (D29). Play desde el principio o desde el punto seleccionado; pausa con rampa y reanudación sin perder la recta del tramo; stop (Esc) que frena y descarta lo pendiente; loop de N veces o infinito; tramo en ejecución resaltado. Los tiempos se respetan aunque la red tenga demoras, porque los ejecuta el reloj del firmware (D6).

Tramo PTP y segmentos temporizados

Implementado en el issue #9. La página de la URC ya lleva el robot a una pose escrita a mano, en J1–J4 o en X, Y, Z y rumbo, con un tramo PTP.

Perfil sincronizado (D53). Todas las articulaciones siguen el mismo perfil de velocidad trapezoidal s(t), de 0 a 1: q(t) = inicio + s(t) · recorrido. Por eso empiezan, aceleran, frenan y llegan a la vez, y el camino es una recta en articulaciones. Los máximos del perfil los fija la restricción más exigente entre las articulaciones del modelo y los motores, con el acople: M2 recorre J1 + J2 (D45). La velocidad del tramo, en porcentaje, reduce las velocidades máximas; la aceleración queda en su máximo.

Segmentos de 20 ms (D27, D50). La URC muestrea el perfil cada 20 ms y envía cada muestra como `SEG n ms q1 q2 q3 q4`. El firmware los encola (16 lugares) y los ejecuta con su reloj, interpolando lineal entre

muestras. Un segmento empieza cuando termina el anterior, aunque haya llegado tarde: una demora del enlace de hasta 320 ms no se nota. La URC envía un `SEG` por vez y espera su `OK`. Con la cola llena, el firmware demora el `OK` hasta que se libera un lugar, así la URC nunca envía de más.

```
URC: perfil s(t) → muestras cada 20 ms → SEG n 20 q1 q2 q3 q4 ─┐
                                                ▲              │
                                              OK n             ▼
UCI: cola de 16 ──→ reloj del firmware ──→ interpolación ──→ pasos de M1–M4
```

Defensas en el firmware (D51). `SEG` exige corriente y robot referenciado, y no se mezcla con el jog. El firmware verifica que ningún motor supere su velocidad en el segmento (`ERR 8`): protege a los motores de un error de la URC. Aceleraciones, límites y choques los verifica la URC antes de enviar.

Paradas (D52). `STOP` y el watchdog frenan con rampa: cada motor sigue en su sentido y baja la velocidad en línea recta hasta cero, todos en el mismo tiempo, con la aceleración del motor más exigido. Desde 90 °/s, J1 frena en 0,64 s y 29°; al 50 % de velocidad global, en 0,32 s y 7°. La cola se vacía y la referencia se conserva. `HALT` para en seco.

Tramo LIN y validación previa

Implementado en el issue #10, como API de Python del planificador (`plan_lin`) y de la validación (`check_lin`, `check_tramo`). El gestor de trayectorias (#12) y la reproducción (#13) la conectan con la página.

Cómo se traza la recta (D54). El punto de herramienta recorre la recta entre los dos puntos, y el rumbo va del de un punto al del otro, con el mismo perfil trapezoidal del PTP. La velocidad del tramo es un porcentaje de 150 mm/s sobre la recta y de 90 °/s de rumbo, la de M1. La componente vertical no pasa de la de J4: una recta vertical al 100 % va a 40 mm/s (D76). En cada muestra de 20 ms la URC resuelve la cinemática inversa con la configuración del codo del tramo y envía el resultado como un `SEG`.

```
recta y rumbo → perfil s(t) → muestra cada 20 ms → cinemática inversa (codo fijo) → SEG
```

Cuánto se aparta de la recta. El firmware interpola lineal en articulaciones entre dos muestras, y la URC redondea cada destino a la centésima. El extremo se aparta de la recta a lo sumo **0,127 mm**:

Efecto	Cálculo	Cota
Redondeo a la centésima	J1 y J2 media centésima de grado cada uno, el antebrazo una entera: $l1 \cdot e + l2 \cdot 2e$	0,066 mm
Cuerda de cada segmento	Brazo y antebrazo giran a lo sumo 1,8° en 20 ms: $(l1 \cdot \Delta 1^2 + l2 \cdot \Delta\varphi^2)/8$	0,062 mm

El plan mide su propio apartamiento: en una recta típica al 50 % es de 0,02 mm. La ejecución suma medio paso de M1 y de M2 (hasta 0,25 mm con 1/8) y el atraso de seguimiento de Webots. Eso lo mide el ensayo.

Qué se verifica antes de arrancar (D14, D55). Cada segmento que se va a enviar, en este orden:

Motivo	Cuándo se rechaza
Límite articular	Un extremo fuera de su recorrido, o el camino lleva J1 o J3 más allá (por ejemplo, al pasar por detrás del robot).

Motivo	Cuándo se rechaza
Cambio de codo	J2 cambia de signo entre los extremos. Para cambiar de lado el brazo tendría que estirarse, que es una singularidad.
Fuera de alcance	La recta pasa a menos de 191 mm del eje de J1, el radio mínimo con J2 a ±135°.
Velocidad excesiva	Una articulación o un motor supera su velocidad o aceleración. Pasa cerca del brazo estirado, donde J2 tiene que girar cada vez más rápido para el mismo avance.
Colisión	El chequeo de colisiones existente, en cada segmento. Por ejemplo, una recta entre dos puntos libres que acerca el soporte del efector a la torre.

El rechazo dice el tramo, el motivo y dónde ocurre, y sugiere PTP. Si el motivo es la velocidad, sugiere primero bajarla. Por ejemplo:

```
Tramo 3 (LIN), velocidad excesiva: M1 (J1) necesitaría aceleración de 145.14 °/s² (máximo
140.00)
al 39 % del tramo. Bajá la velocidad del tramo o usá un tramo PTP.
```

Desde el brazo estirado, una recta casi siempre se rechaza, aun despacio: J2 tendría que arrancar de golpe. Por eso el primer movimiento de un programa es el tramo de aproximación PTP (D32).

> **Evidencia:** en el ensayo de la reproducción (#13), los tramos LIN de una trayectoria en Webots se apartan de la recta a lo sumo 0,21 mm, con la posición estimada y con la física simulada, dentro de la cota de 0,38 mm (0,127 mm del plan más medio paso de M1 y M2). La cota vale para la posición estimada, que está siempre en pasos enteros; la física simulada sigue a los pasos con el atraso del rotor de N2 (D64), y su tolerancia es de 0,46 mm. El PTP y los segmentos tienen su propio ensayo (validación de segmentos). Las trayectorias de demostración, una de ellas un cuadrado LIN de 80 mm, se reproducen en la campaña (D74, sección 10).

Captura de puntos y gestor de trayectorias

Implementado en el issue #12, en la URC (`urc/trajectories.py`) y en un panel de su página. La reproducción (#13) lo usa.

Dónde vive la trayectoria (D62). La URC guarda la trayectoria abierta. La página la muestra y pide cada cambio por HTTP. Así, recargar la página no la pierde, y la reproducción planifica desde los mismos datos.

Teclas y lista (D12, D29, D63).

Acción	Cómo
Capturar	Espacio: la posición estimada por pasos, después del punto seleccionado. Exige robot referenciado y quieto.
Borrar	Retroceso: el seleccionado, que después de capturar es el último capturado.
Terminar	Enter: pide el nombre y guarda.
Editar cómo se llega	En cada fila: tipo de tramo (LIN o PTP), velocidad (%) y espera (s). Nacen LIN, 50 % y 0 s (D22).
Reordenar	Arrastrando la fila, o con ↑ y ↓.

Acción	Cómo
Sobrescribir	Reemplaza la pose del seleccionado por la actual; cómo se llega no cambia.
Ir al punto	PTP al 25 %: muestra la duración y mueve recién con la confirmación.

La lista muestra, por punto, X, Y, Z, rumbo y configuración del codo (D31).

Puntos escritos (D34, D63). Se agregan en X, Y, Z y rumbo o en J1–J4. En XYZ el codo es el preferido (J2 negativo) o el elegido; al editar en XYZ, el punto conserva el suyo. Para cambiarlo se edita en J1–J4. Cada punto se valida al capturarlo, escribirlo, editarlo y cargarlo: alcance, límites articulares y choque en reposo. El rechazo dice qué punto falla y por qué:

```
Punto 3: colisión en reposo, <pieza> puede tocar 0-Torre-1 (envolventes y margen de N mm)
```

Los tramos entre puntos se validan antes de cada play (D55), no al editar: dependen del orden y del tipo.

Archivo (D13, D62). Uno por trayectoria, en la carpeta `trayectorias/`, con «Nueva», «Cargar», «Guardar» y «Guardar como». Si hay cambios sin guardar o el archivo ya existe, la URC pregunta. Cada punto guarda las articulaciones en las unidades de `scara/1`, que son las que valen. También guarda el XYZ y el rumbo derivados, para leerlo, y cómo se llega a él:

```
{
  "format": "scara-trayectoria", "version": 1, "name": "Demostración 1",
  "saved": "2026-10-31T12:00:00-03:00",
  "units": {"joints": "centésimas de grado (J1–J3) y de mm (J4)", "tool": "mm (x, y, z) y grados
(rumbo)",
            "speed": "% de la velocidad máxima", "wait": "s"},
  "points": [
    {"joints": [3000, -6000, 1000, 6000], "tool": {"x": 433.01, "y": 0.0, "z": 34.67, "heading":
-20.0},
     "kind": "LIN", "speed": 50, "wait": 0}
  ]
}
```

Al cargar, la URC verifica el formato, la versión, las unidades y cada punto. Un XYZ que no coincide con las articulaciones se rechaza: el archivo se puede editar a mano, pero no se promociona (D34).

Reproducción

Implementada en el issue #13, en la URC (`urc/playback.py`) y en el panel del gestor.

Antes de mover (D32, D33, D68). Play arranca desde el principio o desde el punto seleccionado, con la velocidad global (10 % a 100 %, no se guarda) y el loop elegidos. La URC planifica y valida todo lo que se va a ejecutar, y recién entonces envía el primer `SEG`:

Tramo	Desde → hasta	Cómo
0, aproximación	Pose actual → punto de partida	PTP al 25 % × la velocidad global, sin espera. Se omite si el robot está a menos de un paso.
2 … n	Punto anterior → punto	Con el tipo, la velocidad × la global y la espera del punto.
Vuelta (loop)	Último punto → punto 1	Con los parámetros del punto 1. Abre cada vuelta nueva.

Si un tramo falla, el mensaje dice cuál («Tramo 0», «Tramo 3», «Tramo de vuelta»), por qué y qué probar. Sin referencia, o con la referencia dudosa por una pérdida de pasos, la reproducción no arranca:

```
Para reproducir, el robot tiene que estar referenciado (homing o referencia manual)
```

Sin huecos entre tramos (D68). Cada tramo se encola entero, con su espera como segmentos sin movimiento, y el siguiente entra cuando queda menos de un segundo por enviar. El firmware los encadena con su reloj: los tiempos son los planificados. La página resalta el punto al que va el tramo en ejecución, que la URC deduce del último segmento terminado que informa el estado `S`.

Pausa, reanudación y stop (D69). La pausa es el `STOP` con rampa del firmware. Al reanudar, la URC recalcula lo que faltaba del tramo desde la pose en la que frenó, con el mismo tipo. En un LIN, la rampa se aparta de la recta lo que se curva el camino en ese trecho, más el escalón de los pasos al frenar en un punto cualquiera (entre 0,05 y 0,16 mm en los ensayos), y el resto es la misma recta. Si el robot se movió durante la pausa, o cambió la trayectoria, hay que empezar de nuevo. Stop y Esc frenan igual y descartan todo lo pendiente.

```
LIN P1 → P2:  ──────────●  pausa: STOP, rampa          reanudar: LIN ●────────── P2
                        (frena sobre la recta)                  (desde la pose de la pausa)
```

Evidencia (ensayo en Webots, corrido en la campaña con los límites de D76: registro). Una trayectoria mixta de 4 puntos (LIN, PTP, un LIN vertical con espera de 1 s), en loop de 2 vueltas desde el reposo:

Qué	Planificado	Medido
Loop completo, con tramo 0 y vuelta	26,86 s	26,86 s (cada tramo dentro de 0,05 s)
Distancia a la recta en los LIN	≤ 0,38 mm la estimada, ≤ 0,46 mm la simulada	0,19 mm (estimada), 0,21 mm (simulada)
Pausa a mitad de un LIN	—	Frena a 0,050 mm de la recta; con la reanudación, el tramo entero queda a 0,21 mm

Los tiempos se miden con el reloj de la simulación, que es el del firmware: ese día Webots corrió a 0,80× del reloj de la PC. Con los límites anteriores (D45), el mismo loop duraba 61,46 s.

9. Qué se simula y qué no

El proyecto simula el diseño; no hay robot físico con el que calibrar (ADR 0001). Cada parámetro declara su procedencia.

Firmware único (D3, ADR 0002). El mismo código C++ corre en el Arduino y dentro del controlador de Webots; solo cambia la capa de hardware. En Webots, la planta simulada reemplaza a drivers, motores, correas y sensores Hall.

Se simula	Cómo
Pasos y microstepping (N1)	Las articulaciones avanzan en pasos discretos

Se simula	Cómo
Transmisión y acople M1/M2 (N1)	4:1, 60T–60T al codo, ACME de 8 mm por vuelta; mover M1 cambia J1 y J2
Par limitado y pérdida de pasos (N2)	El rotor sigue a los pulsos con el par de su curva par–velocidad; si la carga o la aceleración piden más, cae cuatro pasos completos y el firmware no se entera (D64)
Sensores Hall	Imán (`Emitter`) y KY-003 (`Receiver`) del modelo: el Hall se activa por la distancia entre los dos, así que sale de la geometría (D57)
Driver sin corriente	El motor no hace fuerza y sus pasos se pierden; J4 cae por su peso. Al volver la corriente, retiene el eje donde quedó (D60)
Fallas F1–F5	Desde el banco de ensayo, por el canal de instrumentación (D59, D61, D67)
Tiempo	Webots a 1× durante teach y play; el firmware usa el reloj simulado (D26)
Arranque	Cerca de la pose de reposo, con un desvío aleatorio de ±10° y sin referencia

No se simula	Por qué
Modelo eléctrico de los motores (N3): corrientes, inductancia, resonancias	Faltan datos y exigiría pasos de simulación mucho más chicos (ADR 0006)
Encoders o lazo cerrado de posición	El diseño no los tiene
Pinza o herramienta	Fuera de alcance
Botonera física y teclas en Webots	El jog va por la URC; se puede agregar sin cambiar el firmware
Construcción y cableado del robot real	Se entrega firmware que compila para el UNO y la receta de socat

Canal de instrumentación y banco de ensayo (D2, ADR 0004, D59). Lo que solo existe en la simulación viaja por otro puerto (9001) y otro protocolo de texto, `scara-sim/1`: el controlador de Webots envía la posición física simulada, leída de los sensores de posición de Webots, y recibe las fallas, la carga y la recarga del mundo. En la URC, un panel aparte, «Banco de ensayo (solo simulación)», muestra la columna «Simulado» junto a la estimada por pasos y la diferencia. Sin el canal, como con el robot real, el panel no aparece y la URC funciona igual. Capturar y planificar usan siempre la posición estimada: la sesión, el planificador y la validación no conocen el canal, y una prueba lo verifica.

Falla	Qué hace el banco	Qué se ve
F1	Baja la corriente de un driver: el motor tiene menos par	Con 5 kg, J4 no sube y «Simulado» se aparta en múltiplos de 4 pasos completos; al cruzar un Hall, `EV LOST` y la referencia queda dudosa
F2	Deja sin corriente el driver de M4	J4 cae a 120 mm en «Simulado»; la estimada no cambia
F3	Traba un Hall activo o inactivo	El homing aborta (`ERR 12` o `ERR 11`) sin referencia
F4	Corta el enlace `scara/1` sin cerrar la conexión	El estado `S` deja de llegar; el watchdog frena el robot
F5	Reinicia el firmware de la UCI	`BOOT` inesperado: sin referencia y sin corriente, así que J4 cae; J1–J3 quedan donde estaban

Pérdida de pasos (N2, F1; D64–D67). El rotor de cada motor es una inercia que sigue a los pulsos con la curva par–ángulo del motor híbrido. Con un paso completo de atraso da su par máximo, el de la curva

par–velocidad; con dos (medio ciclo eléctrico), cae al equilibrio siguiente y pierde cuatro pasos completos. El firmware sigue contando los pulsos que emitió. Lo descubre al entrar en un imán: con el eje referenciado, el Hall se activa siempre en el mismo flanco, y si la cuenta difiere en 2 pasos completos o más, envía `EV LOST j pasos`. La URC marca la referencia como dudosa hasta un homing. Con la corriente nominal no se pierden pasos dentro de los límites que planifica la URC; para provocarlo, el banco baja la corriente de un driver (F1):

Ensayo en Webots	Qué pasa
Homing y reposo, sin falla	Ningún `EV LOST`; «Simulado» coincide con lo estimado en menos de un paso
5 kg con la corriente nominal	J4 sube y baja a 40 mm/s sin perder pasos
F1 por carga: M4 al 50 % con 5 kg	J4 no sube; al volver al reposo, el Hall de abajo da `EV LOST 4` con los pasos que perdió la planta
F1 por aceleración: M1 al 20 % con 5 kg	El jog de J1, que arranca sin rampa, pierde pasos; al llegar a su imán, `EV LOST 1`. Con ese driver, también la rampa del planificador pierde pasos (D76)
Homing de nuevo	La referencia deja de ser dudosa

Las hojas de datos exactas de los motores no aparecieron: se usan la familia más cercana y supuestos típicos, cada uno declarado (Transmisión).

El canal y las fallas F2–F5 están ensayados en Webots (validación del canal de instrumentación), y F1 y N2, en su propio ensayo (validación de la pérdida de pasos).

10. Validación

La evidencia vigente es la campaña de validación en Webots (issue #18, D73), que reemplaza a la del banco provisional `scara-dev/1`: esa queda como registro del proceso y no se cita como prueba del sistema nuevo (D25). La validación tiene tres niveles, los tres seams de la spec:

Nivel	Qué se prueba	Cómo
Firmware por su protocolo	Modos, homing (también abortado), jog sin referencia, cola y control de flujo, watchdog, acople M1/M2, `EV LOST`	Conversaciones de conformidad contra el núcleo C++ compilado en la PC con una capa de hardware falsa; compilación para el UNO
URC por sus dos bordes	La sesión con la UCI y la interfaz: jog, gestor, reproducción, banco	Las mismas conversaciones contra la URC con una UCI falsa por TCP; endpoints HTTP; el planificador como módulo puro
Escenarios en Webots	El sistema entero con la planta simulada	Ocho ensayos, corridos por un solo ejecutor que registra la versión ensayada

Versión ensayada. El ejecutor compila el controlador y anota el commit y el SHA-256 del controlador compilado, del mundo, del proto y de las trayectorias de demostración; solo registra una campaña completa con el árbol sin cambios (D73). Campaña del 01/11/2026 en Windows 11 con Webots R2025a, microstepping 1/8, planta N2 y los límites de D76: commit `6ddc4ca`.

Escenario	Condición inicial	Resultado	Tolerancia
H1 · Homing desde el reposo	Reposo exacto, sin referencia	J4, J1, J2 y J3 en el flanco de su Hall; vuelta sola al reposo; ningún choque en el camino físico	≤ 3 pasos de 1/8
H2 · Homing con desvío	Desvío al azar de ±10°	Termina igual en el reposo, también después de un homing abortado	≤ 3 pasos; física a menos de un paso completo
T1 · Teach & play	Referenciado	Tres puntos capturados con jog se guardan, se cargan y se reproducen (PTP, LIN, PTP con espera)	Último punto a < 1 paso
T2 · LIN y PTP con loop, pausa y stop	Trayectoria mixta	Cada tramo dura lo planificado (26,86 s medidos contra 26,86 s en 8 tramos); los LIN, a ≤ 0,21 mm de la recta, y tras pausar y reanudar, a 0,21 mm la estimada y 0,16 mm la física	± 0,12 s; ≤ 0,38 mm la estimada y ≤ 0,46 mm la física
T3 · Demostraciones	Las tres de `trayectorias/`	Se reproducen sin errores ni `EV LOST`: barrido PTP en 14,5 s, cuadrado LIN en 10,5 s, *pick and place* en 19,0 s	Último punto a < 1 paso
F1 · Pérdida de pasos	5 kg; un driver con poca corriente	Sin falla, ningún paso perdido; con falla, `EV LOST` con los pasos que perdió la planta y referencia dudosa	± 1 paso completo en J1; ± 2 en J4, que cruza el Hall a 40 mm/s
F2 · Driver sin corriente	J4 en 60 mm	J4 cae a 120 mm; la estimada no cambia	> 118 mm
F3 · Hall trabado	Activo en J4 o inactivo en J1	El homing aborta con `ERR 12` o `ERR 11`, sin referencia	Sin referencia
F4 · Enlace cortado	Jog en marcha	El watchdog frena en 1 s; al volver, el robot está quieto	J1 quieto 2,5 s después
F5 · Reinicio de la UCI	Quieto o en plena reproducción	`BOOT` inesperado, sin referencia; la reproducción se detiene y no se puede volver a dar play	Un `BOOT`

Los tres escenarios de soporte, protocolo `scara/1` (P1), transmisión N1 (P2) y segmentos con PTP (P3), también pasan. El detalle, con los registros, está en el informe de la campaña.

El jog verificado tiene su propio ensayo. El teach de la campaña (T1) mueve el robot con órdenes `JOG` de `scara/1`, no con el jog cartesiano y verificado de la URC (issue #11, D75). El jog de la URC tiene su ensayo en Webots, con los resultados de la sección 8, y sus pruebas por los endpoints HTTP en la suite.

Hallazgos de la campaña.

- La física simulada se aparta de la recta un poco más que la estimada: sigue a los pasos con el atraso del rotor de N2. Por eso la tolerancia de la física es de 0,46 mm y la de la estimada, de 0,38 mm.
- A 40 mm/s, el Hall de J4 se lee con 2 pasos completos de error: entre dos pasos de Webots (2 ms), J4 avanza 0,08 mm. El `EV LOST` dio 1254 pasos contra 1252 que perdió la planta.
- La pausa no siempre frena a 0,05 mm de la recta, como en el ensayo de la reproducción (D69): en la campaña quedó a 0,164 mm, por el escalón de los pasos al frenar. Queda dentro de la misma tolerancia.

- Dos ensayos tenían esperas frágiles, y se corrigieron sin aflojar lo que miden.

Pruebas automáticas (seams 1 y 2). `python -m unittest discover` corre, sin Webots, las conversaciones de conformidad contra el firmware y contra la URC, la compilación para el UNO, los endpoints HTTP del jog, el gestor, la reproducción y el banco, el planificador, el modelo N2, los flancos de los Hall contra la geometría del proto, las tres demos y la forma del registro de la campaña. En la versión de este documento: 196 pruebas en verde; se omiten 3 de propiedades CAD, que necesitan el Python con OCP.

Trayectorias de demostración (D74). Tres, versionadas en `trayectorias/`, la carpeta por defecto de la URC: *pick and place* (LIN para bajar y subir, PTP para cruzar, esperas), un cuadrado LIN de 80 mm y un barrido PTP con cambio de codo. Una prueba las carga y las reproduce en loop de 2 vueltas en cada corrida de la suite.

Límites. Solo se validó en Windows; macOS queda pendiente (D19). El firmware para el UNO compila y su camino serie se probó con un puerto virtual y socat, pero no con la placa (D49). Los ensayos corren con Webots minimizado: lo visual queda para la QA humana. Los parámetros siguen siendo provisionales: la campaña prueba el diseño simulado, no un robot construido (ADR 0001).

11. Registro de decisiones

Las 76 decisiones de la ronda, cada una con su porqué en una línea. El texto completo, con qué se descartó, está en el registro de decisiones. Las difíciles de revertir tienen un ADR. Todas están aceptadas, salvo las dos propuestas al profesor (D9 y D10). ADR 0001, simular el diseño sin hardware, es anterior al registro y enmarca todo el documento.

Alcance, arquitectura y protocolo

D	Decisión	Por qué	ADR
D0	Teach & play, homing y fallas sobre la simulación existente; AGSABOT como idea	Es lo que pidió el profesor; AGSABOT reproduce a ciegas cada 1200 ms, su stop puede no enviarse y no tiene homing	
D1	La URC es servidor TCP y la UCI se conecta	Lo pidió el profesor; con el robot real, socat abre el USB y es quien se conecta; el robot puede reiniciarse sin tocar la URC	
D2	Canal de instrumentación aparte	La URC no puede depender sin querer de lo que el robot real no da	0004
D3	Un único firmware C++ para Arduino y Webots	El día del robot real no hay que desarrollar nada nuevo	0002
D4	Una línea por mensaje, LF, hasta 64 bytes, solo enteros	Se procesa en 2 KB de RAM y se escribe a mano con `nc` o el monitor serie	
D5	El protocolo habla en unidades articulares	Poleas, microstepping y acople quedan en el robot: cambiar un driver no toca la URC	0003
D6	La URC planifica y el firmware ejecuta, con su reloj	El UNO no calcula la inversa a la frecuencia de una recta; el reloj del firmware evita los tirones por la red	0005
D7	Nombres UCI y URC	Es el vocabulario que la cátedra conoce de AGSABOT	
D24	Puertos 9000 y 9001; versión en `BOOT` y `HI`	Configurables, y la URC sabe con qué firmware habla	
D26	Webots a tiempo real durante teach y play	Con tiempo simulado más rápido, el watchdog saltaría y «respetar los tiempos» perdería sentido	
D27	Números iniciales: `S` a 20/s, PING cada 250 ms, watchdog de 1 s, segmentos de 20 ms, cola de 16	Caben con holgura en un UNO a 115200 baudios	

Mecánica, electrónica y motores

D	Decisión	Por qué	ADR
D8	Modelo de motores N1 + N2, sin N3	Muestra lo que distingue a un paso a paso, incluida una falla que el firmware no advierte; N3 exige datos que no hay	0006

D	Decisión	Por qué	ADR
D9	**Propuesta al profesor:** microstepping 1/8 (1600 pulsos/vuelta)	Con 400 pulsos, un pulso de M1 mueve el extremo 2 mm y las rectas se ven escalonadas; con 1/8, 0,5 mm, y el UNO alcanza	
D18	Arduino UNO por USB + socat; un Hall por articulación	Es la placa del maestro, y con un Hall por eje entran 15 de 20 pines	
D20	2 NEMA 23 (M1, M2) + 2 NEMA 17 (M3, M4)	Coinciden profesor, maestro y planos; el `NEMA23-2` del ensamble no tiene ubicación	
D44	El microstepping es un solo parámetro, en `firmware/config.h`	Si la planta tuviera su propio valor, el robot simulado se movería distinto de lo que cree el firmware sin que falle una prueba	
D45	Límites de velocidad y aceleración por motor, derivados a las articulaciones con el acople	La limitación física está en los motores; con el acople, J2 no tiene un límite propio. Los valores son de D76	
D46	Jog por motor: `JOG Mm ± vel`	Muestra en la defensa que mover M1 cambia J1 y J2 y no el rumbo, como señaló el profesor	
D47	Pines del UNO compatibles con el shield Ethernet	STEP en un mismo puerto: los pulsos de M1 y M2 salen juntos; pasar a Ethernet no obliga a recablear	
D48	El núcleo cuenta los pasos y el Timer1 los convierte en pulsos	Los tiempos del TB6600 no dependen del lazo, y nunca se descarta un paso contado	
D49	Ensayo del camino serie con una UCI de PC detrás de un puerto virtual	Prueba sin placa la receta de socat y la URC sin cambios	

Referencia, homing y Hall

D	Decisión	Por qué	ADR
D10	**Propuesta al profesor:** un Hall por articulación con dos imanes; J2 relativo; referencias de J1 y J2 en el extremo negativo, J4 arriba	Los topes que importan son los relativos; con el mismo signo en J1 y J2, el homing no choca con la torre	
D11	Secuencia de homing J4, J1 (M1 y M2), J2 (M2), J3: rápido, retroceso y lento	Subir J4 aleja la herramienta; mover M1 y M2 juntos fija J2; llegar lento desde el mismo lado da repetibilidad	
D21	Sin referencia, solo jog articular lento; capturar y reproducir bloqueados	Sin referencia la posición es desconocida: el robot real solo conoce sus pasos	
D30	Pose de reposo: brazo estirado, J4 abajo	Contraída toca la torre aun quieta; estirada queda libre y J1 barre hasta los dos extremos	
D31	Configuración del codo preferida: J2 negativo	El brazo estirado es una singularidad: hay que elegir una de las dos soluciones de forma predecible	
D56	Homing por flanco en cinco fases; la URC planifica la vuelta al reposo	El flanco al que se llega lento es repetible y queda 5° antes del tope; la vuelta verifica choques como todo PTP	
D57	En Webots, el Hall se activa por la distancia entre imán y sensor	Webots no entrega paquetes dentro de un mismo robot; la distancia es la misma regla y sale de la geometría	

D	Decisión	Por qué	ADR
D58	Sin referencia, jog a 5 °/s y Hall como fin de carrera	Es D21 en el firmware: sin referencia, nadie más protege el robot	
D72	Homing más rápido (15 °/s; 10 mm/s en J4) y repetible después de frenarlo	Tardaba 80 s; N2 verifica que esas velocidades no pierden pasos; con `STOP` la cuenta sigue valiendo	

Operación: jog, trayectorias y reproducción

D	Decisión	Por qué	ADR
D12	Jog con el teclado de la URC, cartesiano y articular	Funciona igual con el robot real, que no tiene botonera, y la lista de puntos queda a la vista	0007
D13	Las trayectorias las guarda la URC, como JSON, con la pose y no el camino	El robot solo informa la posición; el camino se calcula al reproducir	
D14	Tramos LIN o PTP; un LIN imposible se rechaza antes de arrancar	El profesor pidió recta o menor recorrido; frenar a mitad de camino deja al robot donde no se planeó	
D22	Tramos por defecto LIN, 50 %, 0 s; velocidad global de 10 a 100 %	Primero se enseñan posiciones y después se ajustan velocidades; la global permite probar despacio	
D28	Jog verificado una vez referenciado	El jog no debe poder llevar el robot contra la torre	
D29	Funciones del gestor de trayectorias	Lo útil de AGSABOT más lo que le faltaba: stop real, pausa, progreso y validación previa	
D32	Tramo de aproximación PTP; los parámetros del punto 1 rigen la vuelta del loop	El punto de partida no está grabado; un PTP entre dos poses alcanzables siempre se puede hacer	
D33	Tramo de aproximación al 25 % × la velocidad global	Es el único movimiento desde una pose no grabada: despacio da tiempo a frenar con Esc	
D34	Puntos escritos en XYZ y rumbo o en J1–J4, validados	Con jog es difícil clavar un valor exacto; un punto escrito nunca fue visitado	
D50	Un `SEG` sin confirmar por vez; con la cola llena, `OK` demorado	Control de flujo de GRBL: la URC nunca desborda la cola y el enlace se sigue leyendo	
D51	El firmware valida la velocidad de cada motor en cada `SEG`	Protege a los motores de un error de la URC al costo de una cuenta por segmento	
D52	`STOP` y el watchdog frenan con rampa en la dirección del segmento	Frenar en seco desde plena velocidad pide aceleración infinita y pierde pasos	
D53	Tramo PTP: un perfil trapezoidal común, limitado por articulaciones y motores	Todas las articulaciones empiezan y llegan a la vez, como en el PTP industrial	
D54	Tramo LIN: velocidad del punto de herramienta y tolerancia calculada (0,127 mm desde D76)	Como en el LIN industrial, el operador elige la velocidad del extremo; la cota vale para todo tramo validado	
D55	Validación previa de cada segmento que se va a enviar	Cubre el redondeo y las muestras, no solo la recta ideal; el rechazo dice qué corregir	

D	Decisión	Por qué	ADR
D62	La trayectoria vive en la URC; en el archivo `scara-trayectoria` mandan las articulaciones	Recargar la página no la pierde; las articulaciones guardan el codo y las vueltas sin redondear	
D63	Capturar con el robot referenciado y quieto; el punto escrito conserva su codo y se valida	En movimiento, el punto capturado no sería donde el robot se detiene	
D68	Validar todo antes de mover y encolar un tramo detrás de otro, con sus esperas	El robot nunca arranca un tramo que no puede terminar, y el firmware los encadena sin huecos	
D69	La pausa es el `STOP` con rampa; al reanudar se recalcula el resto del tramo	El firmware ya frena así y vacía la cola al instante; el resto de un LIN es la misma recta	
D70	Con referencia, jog a 10 °/s, frenado por la URC antes del límite y del choque	A 5 °/s no servía para mostrar, y con referencia el jog no verificaba nada. Desde D75 rige solo para el jog por motor	
D71	Un PTP puede salir de la zona de margen de un obstáculo si se aleja	Desde muy cerca, todo camino se rechazaba y el robot quedaba encerrado	
D75	Con referencia, el jog son segmentos de 20 ms planificados de a uno, tres en cola contados en el reloj de la UCI	Es D28 con el cartesiano; el robot frena en 60 ms sobre el último segmento verificado	

Canal de instrumentación, pérdida de pasos y fallas

D	Decisión	Por qué	ADR
D23	Catálogo de fallas F1–F6 (F6 opcional)	Son fallas que el robot real puede sufrir, y la mayoría se detectan sin encoders	
D59	`scara-sim/1` por el puerto 9001; la URC solo lo muestra	Es lo que pide el ADR 0004; un protocolo de texto se lee a mano y no necesita JSON en C++	0004
D60	Sin corriente, el motor no hace fuerza y pierde los pasos; al volver, retiene el eje donde quedó	Es lo que hace un paso a paso real: J4 cae y la posición estimada deja de valer	
D61	F4 corta el enlace sin cerrar la conexión; F5 reinicia solo el firmware; la carga es `toolMass`	Como un cable desenchufado; la URC tiene que detectar el `BOOT` con el robot en la misma pose	
D64	N2: el rotor sigue al campo con la curva par–ángulo y el par de su curva par–velocidad	Pierde pasos de a cuatro, como un paso a paso real; el control de posición de Webots daba falsas pérdidas	0006
D65	`EV LOST` al entrar en un imán, con 2 pasos completos de tolerancia, sin corregir la cuenta	El Hall es la única medida absoluta; corregir en movimiento escondería la falla	
D66	N2 confirma los límites de D45. Reemplazada por D76	Con la corriente nominal y 5 kg, el par pedido quedaba entre 1,4 y 22 veces por debajo del disponible	
D67	F1 baja la corriente de un driver: `F1 m porcentaje`	Un TB6600 regulado de menos es una falla real, y muestra la pérdida por carga y por aceleración	
D76	Motores más rápidos, con el doble del par que piden; el LIN se separa de J4	El margen de D66 sobraba y la demostración era lenta: un PTP de 90° de J1 pasa de 5,1 a 1,6 s; 2 veces cubre lo que N2 no modela	

Bala trazadora y protocolo completo

D	Decisión	Por qué	ADR
D35	Forma de `BOOT`, `HI`, `S` y `ERR`	El modo como palabra se lee a mano; el código numérico deja decidir a la URC sin comparar textos	
D36	En Webots, el firmware arranca con la primera conexión	Imita al UNO, que se reinicia cuando socat abre el puerto; un corte de red no pierde la referencia	
D37	Tres frenos para el jog: la página, la URC y el watchdog	Cada uno puede caerse por separado	
D38	Mundo nuevo al lado del viejo, que arranca en la pose de reposo	El banco viejo sigue funcionando y `ZERO` es verdad en la simulación	
D39	Pruebas del firmware como conversaciones	Se prueba el comportamiento por el protocolo, dentro de `unittest`	
D40	La UCI arranca sin corriente; `DISABLE` pierde la referencia	Sin corriente el motor gira libre; tras un reinicio inesperado, el robot no retoma nada solo	
D41	`STOP` conserva la referencia; `HALT` la pierde si había movimiento	Es la división de GRBL entre *feed hold* y *reset*: parar en seco puede perder pasos	
D42	Conformidad simétrica: las mismas conversaciones contra el firmware y la URC	Los dos lados no pueden entender el protocolo de maneras distintas	
D43	Todo `BOOT` deja al robot sin referencia en la URC	Las órdenes pendientes eran para un firmware que ya no existe	

Proceso y entrega

D	Decisión	Por qué	ADR
D15	Interfaz web local servida por la URC	Ya existe, no agrega dependencias y la consigna admite una página web	
D16	El camino nuevo se construye al lado del viejo	Nunca quedarse sin algo que funcione	
D17	Maestro v2 en Markdown, exportado a PDF, sin tocar el original	Se versiona con el código y se exporta para clase	
D19	Compilación en Windows, validada en la bala trazadora	Resuelta el 30/10/2026: compila con el `g++` de Webots. macOS queda pendiente	
D25	Documentación y evidencia del camino viejo a histórico	Una sola descripción vigente; lo viejo no se cita como prueba del sistema nuevo. Lo hace el issue #1	
D73	Un ejecutor corre los ensayos de cada issue y registra la versión ensayada	Reusar los ensayos evita dos versiones de la misma prueba; el SHA-256 identifica el binario	
D74	Las trayectorias de demostración se versionan en `trayectorias/`	Aparecen en «Cargar» para la defensa, y la suite avisa si una deja de valer	